DS605 · FUNDAMENTALS OF MACHINE LEARNING

Lecture Notes

Evaluation — II

Loss Functions

Lecture 11–12
25 August 2026

🐕🐱🐼 **Threading example:** *one dog / cat / panda class-balance prior powers the entropy, cross-entropy, and KL divergence worked examples*

Contents

1. Where This Fits 3
2. From Error Rate to Loss 3
3. Defining the Loss Function 3
4. Generalization Loss and Empirical Loss 3
5. Choosing a Loss Function 4
6. Mean Squared Error (L2 Loss) 4
7. Mean Absolute Error (L1 Loss) 4
8. Huber Loss: Best of Both Worlds 5
9. Entropy: Measuring Uncertainty From Scratch 5
10. Cross-Entropy 6
11. Categorical, Binary, and Multi-label Cross-Entropy 7
12. Kullback–Leibler Divergence 8
13. Other Classification Losses: Hinge Loss 9
14. Loss Function Cheat-Sheet 9
15. Key Takeaways 9
16. Looking Ahead: Bridge to Optimization 10

1. Where This Fits

This lecture is the second half of Evaluation, and it sits between two others in the course arc:

Lectures	Theme
09–10	Evaluation I — data splits, resampling, metrics
11–12 (this lecture)	Evaluation II — how do we weigh different mistakes?
13	Optimization — how do we search for a good hypothesis?

2. From Error Rate to Loss: Not All Mistakes Cost the Same

Misclassification rate treats every mistake identically. But in practice, it's often worse to make one kind of mistake than another. Consider two spam filters: it is worse to classify non-spam as spam than to classify spam as non-spam — losing a real email is more costly than a little junk mail slipping through. Which model would you actually deploy?

Model A — 1% error rate

Mostly misclassifies spam as non-spam (a little junk mail slips through).

Model B — 0.5% error rate

Mostly misclassifies non-spam as spam (real mail gets buried in the spam folder).

Model B has the lower error rate — but if losing real mail is the costlier mistake, Model A may well be the better deployment choice. Error rate alone can't tell them apart; we need a function that assigns a cost to each kind of mistake. That function is the loss function, and it is the subject of this lecture.

3. Defining the Loss Function

The loss *L(x, y, ŷ)* is the amount of utility lost by predicting *h(x) = ŷ* when the correct answer is *f(x) = y*:

$$L(x,\ y,\ \hat{y}) \ = \ Utility(f(x)) \ - \ Utility(h(x))$$

This is often simplified to *L(y, ŷ)*, independent of *x*.

We choose the learner that minimizes the expected loss over all input–output pairs it will ever see, not the learner that minimizes the raw error count.

4. Generalization Loss and Empirical Loss

Let *E* be the set of all possible input–output examples, drawn from a prior distribution *P(X, Y)*. The expected **generalization loss** of hypothesis *h* is:

$$GenLoss_L(h) = \sum_{(x,y) \in E} L(y, h(x)) \cdot P(x,y)$$

Because *P(X, Y)* is unknown, we can only estimate it with the **empirical loss** on a sample *D* of size *N*:

$$EmpLoss_{L,D}(h) = \sum_{(x,y) \in D} L(y, h(x)) \cdot \frac{1}{N}$$

We then pick the hypothesis that minimizes empirical loss over the hypothesis space:

$$\hat{h}^* = argmin_{h \in H} EmpLoss_{L,D}(h)$$

Four reasons the learned $\hat{h}^*$ may still differ from the true target function f:

Reason	What it means
Unrealizability	f itself may not lie anywhere in the hypothesis space H.
Variance	A different training sample would have produced a different $\hat{h}^*$.
Noise	The labels y themselves may be imperfect or stochastic.
Computational complexity	The optimizer may not find the true minimizer of empirical loss.

5. Choosing a Loss Function

The right choice depends on the learning algorithm, how easily its derivative can be computed, and the share of outliers in the data. Commonly used losses fall into two families:

Family	Used for
Regression losses	Continuous targets (MSE, MAE, Huber)
Classification losses	Categorical targets (cross-entropy, hinge, KL divergence)

6. Mean Squared Error (L2 Loss)

$$MSE = \frac{1}{n} \sum_{x \in S} (h(x) - y)^2$$

Almost every data scientist's default choice for regression — most variables can be reasonably modeled as Gaussian, and MSE is fully differentiable everywhere, which makes it friendly to gradient-based optimization (a preview of the next lecture).

Writing e = h(x) − y for the prediction error: the gradient of MSE grows linearly with e. A point that's twice as wrong pulls the model twice as hard — which is exactly what makes MSE so sensitive to outliers.

7. Mean Absolute Error (L1 Loss)

$$MAE = \frac{1}{n}\sum_{x \in S} |h(x) - y|$$

One of the simplest yet most robust loss functions. Real-world regression targets are rarely strictly Gaussian — outliers (values far from the rest) can dominate MSE. MAE is the better option in those cases.

MAE has a constant gradient magnitude — an outlier pulls no harder than a near-miss. The cost: the derivative is undefined at e = 0, so optimizers use a subgradient there.

8. Huber Loss: Best of Both Worlds

MSE handles the common case well but panics at outliers; MAE ignores outliers but has a kink at zero. Huber loss blends the two, switching behavior at a threshold δ:

$$L_\delta(y, f(x)) = \{\frac{1}{2}(y - f(x))^2 \; for \; |y - f(x)| \le \delta \; \delta \cdot |y - f(x)| - \frac{1}{2}\delta^2 \; otherwise$$

WHERE YOU'LL SEE THIS

Robust regression — financial/insurance modeling, sensor data — anywhere a few extreme, legitimate outliers shouldn't be allowed to dominate training.

Reinforcement learning — the DQN algorithm trains its Q-value network with Huber loss so that occasional large temporal-difference errors don't destabilize learning.

Worked example — two prediction errors, both scored with δ = 1:

Error e = y − f(x)	|e| vs δ	Branch used	Loss
e = 0.5 (small)	0.5 ≤ 1	quadratic	½(0.5)² = 0.125
e = 2.5 (outlier)	2.5 > 1	linear	1·2.5 − ½(1)² = 2.000

Watching the gradient across this threshold: inside [−δ, δ] it still rises linearly like MSE's gradient; past it, it flattens to a constant ±δ like MAE's — capping how hard any single outlier can pull on the model.

9. Entropy: Measuring Uncertainty From Scratch

Before cross-entropy, we need entropy itself. Imagine a biased coin, heads with probability p. Always-heads carries no surprise — you already know the outcome. A fair coin is the opposite: every flip is a toss-up. Entropy turns that intuition into a number:

$$H(p) = -[p \cdot log_e(p) + (1-p) \cdot log_e(1-p)]$$

Worked example — suppose our image dataset's class balance is dog 50%, cat 30%, panda 20%:

Class	p(x)	$\log_e$ p(x)	$-p(x) \cdot \log_e$ p(x)
Dog	0.5	−0.693	0.347
Cat	0.3	−1.204	0.361
Panda	0.2	−1.609	0.322

Summing the last column gives H(P) = 0.347 + 0.361 + 0.322 = 1.030 nats. A uniform 3-way split (⅓, ⅓, ⅓) would give the maximum possible entropy for 3 classes, $\log_e 3 \approx 1.099$ nats. Our distribution is a little more predictable than that, since "dog" dominates.

10. Cross-Entropy

Entropy H(P) is the minimum average number of nats needed to describe outcomes from a true distribution P, using a code built for P. Cross-entropy asks: what if we used a code built for a different, believed distribution Q instead — here, P = target, Q = prediction?

$$H(P,Q) = -\sum_x P(x) \cdot log_e(Q(x))$$

Using the wrong code is never cheaper than the right one, so H(P, Q) ≥ H(P) always — with equality exactly when Q = P.

Meet the Example: Dog, Cat, or Panda?

A classifier looks at one image and must decide which of three classes it belongs to. For every image it outputs a full probability vector over all three classes — and the true label is written the same way, as a vector. This photo is of a dog. As a one-hot target vector: P = [1, 0, 0] over (dog, cat, panda). The model doesn't see this vector — it only sees the photo, and must output its own guess Q.

Cross-Entropy Loss Function

The model outputs a probability vector over all classes, summing to 1 — its guess Q for our dog photo:

$$L = -\sum_{x} p(x) \cdot log(q(x))$$

where p(x) is the probability of class x in the target, and q(x) is the probability of class x in the prediction. This reuses the same 0.5 / 0.3 / 0.2 numbers as the entropy example — there, they described a dataset's class balance; here, they describe a model's belief about a single photo. Same formula, different role for the numbers.

Worked example — image contains a dog. Target is the one-hot vector from above: dog = 1, cat = 0, panda = 0.

Class	Target p(x)	Prediction q(x)	Loss for class
Dog	1	0.5	−1·log 0.5 = 0.693
Cat	0	0.3	0
Panda	0	0.2	0

Every non-hot term vanishes — only the probability the model assigned to the correct class matters. That observation is exactly where categorical cross-entropy (Section 11) comes from.

How Cross-Entropy Behaves

The further the predicted probability of the true class is from 1, the faster the loss grows, approaching infinity as the prediction approaches 0. Notice q = 0.5 gives loss ≈ 0.693, the same value as the fair-coin entropy from Section 9 — it's the same logarithm, evaluated the same way.

11. Categorical, Binary, and Multi-label Cross-Entropy

Categorical Cross-Entropy

If the target is a one-hot vector, every non-hot class contributes exactly zero — exactly what happened with dog/cat/panda above — so we can forget them entirely and score only the hot class:

$$L = -log(q(x))$$

Binary Cross-Entropy

A specific case of cross-entropy where the target is 0 or 1 — e.g., "does this image contain a panda?"

$$L = -(p(x) \cdot log(q(x)) + (1 - p(x)) \cdot log(1 - q(x)))$$

Image	Target	P(panda)	Loss
Panda photo	1	0.8	−log 0.8 = 0.223
Not a panda	0	0.6	−log(1−0.6) = 0.916

Equivalent to categorical cross-entropy on the one-hot pair [p(x), 1 − p(x)] — this closed form just skips that conversion.

Multi-label Classification

The target can represent multiple classes at once — or none. Target and prediction are not probability vectors here; each class is its own independent yes/no question.

Class	Target	Prediction
Dog	1	0.6
Cat	0	0.7
Panda	1	0.4

Image contains a dog and a panda, but no cat.

Worked example — treat this as three independent binary classification subtasks, one binary cross-entropy per class:

Class	Which term applies	Binary cross-entropy
Dog	target = 1 ⟶ −log(q)	−log 0.6 = 0.511
Cat	target = 0 ⟶ −log(1−q)	−log 0.3 = 1.204
Panda	target = 1 ⟶ −log(q)	−log 0.4 = 0.916

Compute the binary cross-entropy for each class separately, then sum:

$$L = \sum_{x} L_{binary}(x)$$

12. Kullback–Leibler Divergence

Cross-entropy told us H(P, Q) ≥ H(P). The gap between them — the extra nats wasted by using the wrong code Q instead of the true P — is its own quantity, the KL divergence:

$$D_{KL}(P\|Q) = \sum_{x} P(x)\cdot log\frac{P(x)}{Q(x)} = H(P,Q) - H(P)$$

Worked example — true class balance P = (0.5, 0.3, 0.2) — the same dog/cat/panda prior from the entropy slide — against a model that believes Q = (0.4, 0.4, 0.2):

Class	P(x)	Q(x)	P(x)·log[P(x)/Q(x)]
Dog	0.5	0.4	0.5·log(1.25) = 0.112

Class	P(x)	Q(x)	P(x)·log[P(x)/Q(x)]
Cat	0.3	0.4	0.3·log(0.75) = −0.086
Panda	0.2	0.2	0.2·log(1) = 0.000

Summing gives D_KL(P‖Q) = 0.112 − 0.086 + 0.000 = 0.025 nats. This ties back to Section 9's worked example, and cross-checks via H(P, Q) − H(P): H(P, Q) = 1.055, and we already computed H(P) = 1.030. 1.055 − 1.030 = 0.025 — matches exactly.

APPLICATIONS

Variational Autoencoders — regularizes the learned latent distribution to stay close to a simple prior (e.g., a standard normal), so the latent space stays smooth and sampleable.

Knowledge distillation — a small "student" network is trained to match a large "teacher" network's soft output distribution, not just its hard label.

Policy-gradient RL — trust-region methods (TRPO, PPO) cap how far a new policy is allowed to drift from the old one, measured in KL divergence, for stable updates.

Variational / Bayesian inference — measures how well a tractable approximate posterior matches the true (often intractable) posterior distribution.

Because it's asymmetric, which direction you minimize matters: DKL(P‖Q) tends to spread Q to cover all of P's mass; DKL(Q‖P) tends to let Q lock onto just one mode of P.

13. Other Classification Losses: Hinge Loss

Used specifically in Support Vector Machines, with labels encoded as y ∈ {−1, +1}. Unlike cross-entropy, hinge loss doesn't just penalize misclassifications — it penalizes anything inside the decision margin, even if correctly classified.

$$L = max(0, 1 - y \cdot f(x))$$

Case	y · f(x)	Loss
Confidently correct	1.5	max(0, 1−1.5) = 0
Inside the margin	0.3	max(0, 1−0.3) = 0.7

14. Loss Function Cheat-Sheet

Loss	Best for	Key property
MSE (L2)	Regression, Gaussian-like targets	Differentiable, outlier-sensitive
MAE (L1)	Regression with outliers	Robust, non-differentiable at 0
Huber	Regression, mixed noise, RL (DQN)	Quadratic near 0, linear far away
Categorical CE	Single-label, multi-class classification	Scores only the true class's probability
Binary CE	Yes/no classification	Special case of CE with 2 classes
Multi-label CE	Several independent labels per item	Sum of per-class binary CE
Hinge	SVMs	Margin-based
KL Divergence	VAEs, distillation, RL, variational inference	Asymmetric, ≥ 0, zero iff P=Q

15. Key Takeaways

- A loss function encodes which mistakes cost more — error rate alone cannot.
- MSE, MAE, and Huber trade off Gaussian-fit, outlier-robustness, and a tunable blend of both.
- Entropy measures uncertainty in one distribution; cross-entropy scores a prediction against a target using the same formula.
- KL divergence is the gap between them — cross-entropy minus entropy — and the standard way to match two distributions.

16. Looking Ahead: Bridge to Optimization

COMING UP · LECTURE 13

Representation ✓ → Evaluation ✓ → Optimization (next)

We can now measure how wrong a model is — and we've plotted the gradient of every loss along the way. Next: how do we actually search for the model that minimizes it?